Performed by: Vipul Jadhav

Assignment 6

Task: Deploy a WAR application from a Jenkins Master to a separate EC2 instance that is not configured as a Jenkins slave. The Jenkins Master contains the WAR file, while the target EC2 instance has an Apache Tomcat server installed. Using a Jenkins job with a shell script, the Jenkins Master connects to the remote instance via SSH, transfers the WAR file, and deploys it to the Tomcat webapps directory to make the application accessible.

Steps:

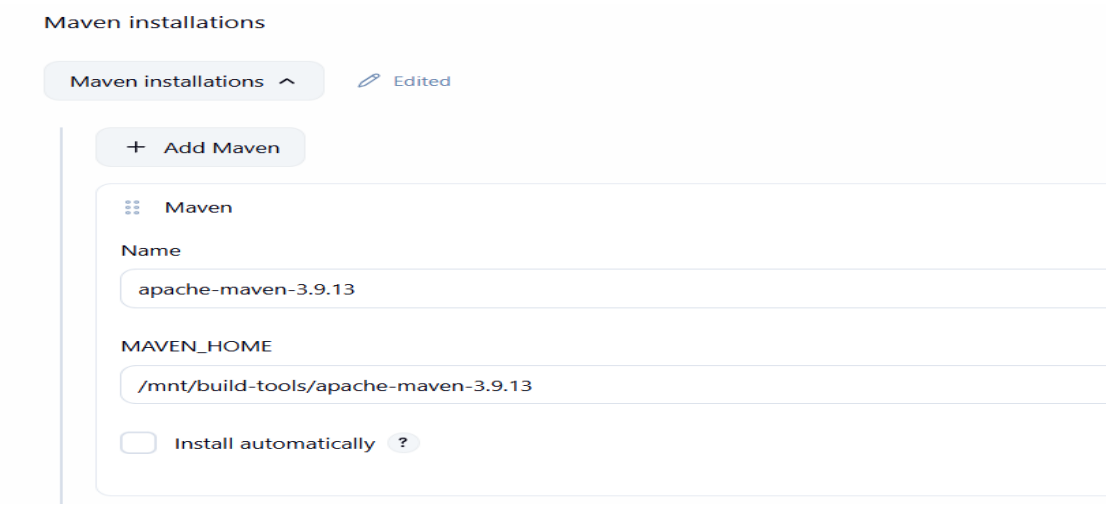
1. Launched 2 ec2 instances

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
jenkins-master	i-03c64cb8de1ab193c	Running	t3.micro	3/3 checks passec	View alarms +	ap-south-1a
inst-a	i-0965b55b0232d41d5	Running	t3.micro	3/3 checks passec	View alarms +	ap-south-1a

2. Installed java, tomcat v10, Jenkins and maven in Jenkins master manually
3. Created environment variable for maven

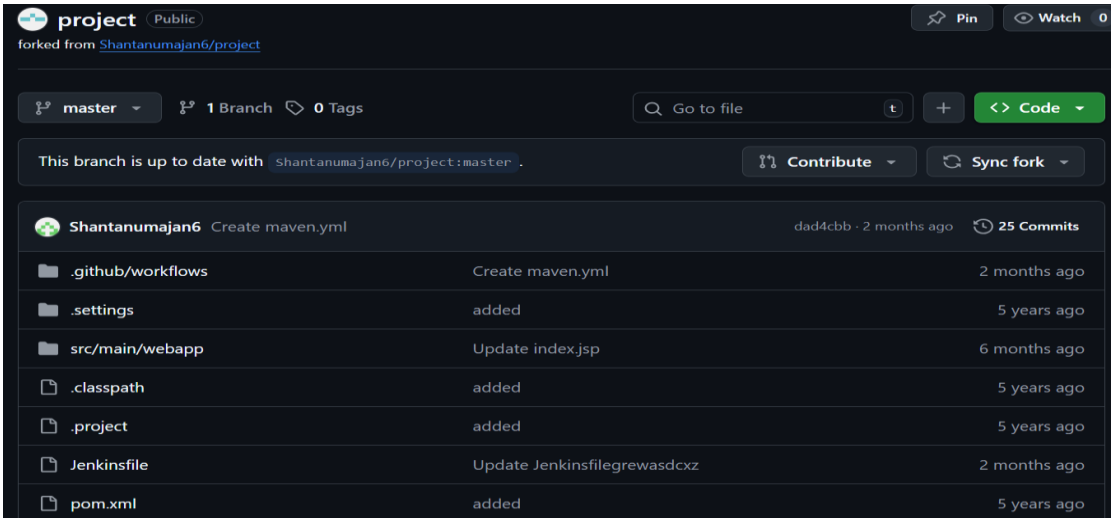
```
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
        . ~/.bashrc
fi

# User specific environment and startup programs
#
#
export MAVEN_HOME=/mnt/build-tools/apache-maven-3.9.13
export PATH=$MAVEN_HOME/bin:$PATH
```

4. In Jenkins tools section uncheck install maven automatically checkbox and add name of maven (added apache-maven-3.9.13) and add the location of maven home.



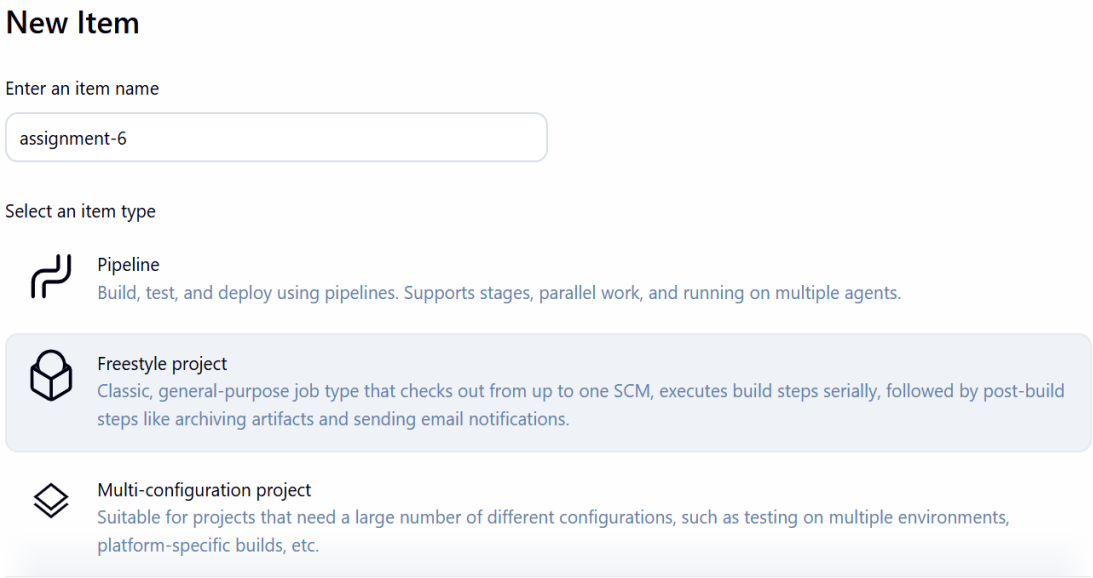
5. Created Jenkins workspace directory (/mnt/project) in master instance

6. Github repository that we are going to use.

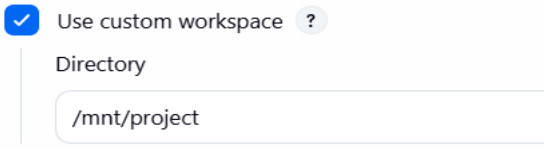


7. Copied key pair manually in Jenkins master (in /mnt/other directory) to access other ec2 instance through Jenkins.

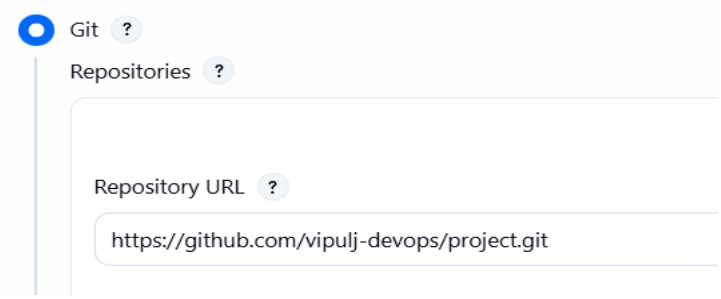
8. Created freestyle project with the name of assignment-6 in Jenkins



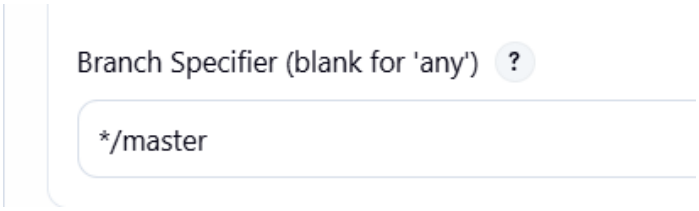
9. Added custom workspace



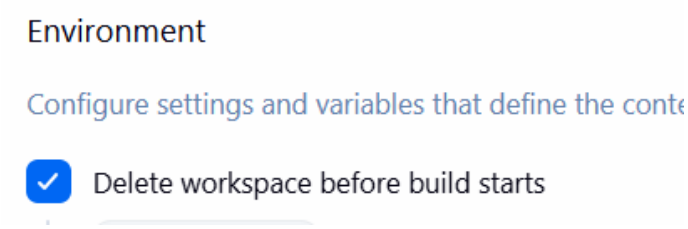
10. Put GitHub repository URL in GitHub URL section of Source Code Management



11. Select master branch in Branch Specifier

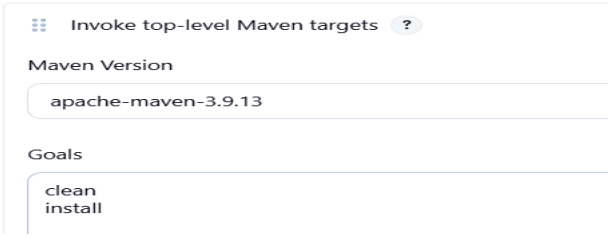


12. Check on 'Delete workspace before build starts' option. So, every time when we start new build the workspace will be cleared



13. Now in build step wrote first shell script to clean the maven repository

14. In build step selected Invoke top-level Maven targets configured Maven version and specified the goal in which mentioned two maven tasks (clean and install)



15. Wrote the below shell script in execute shell section

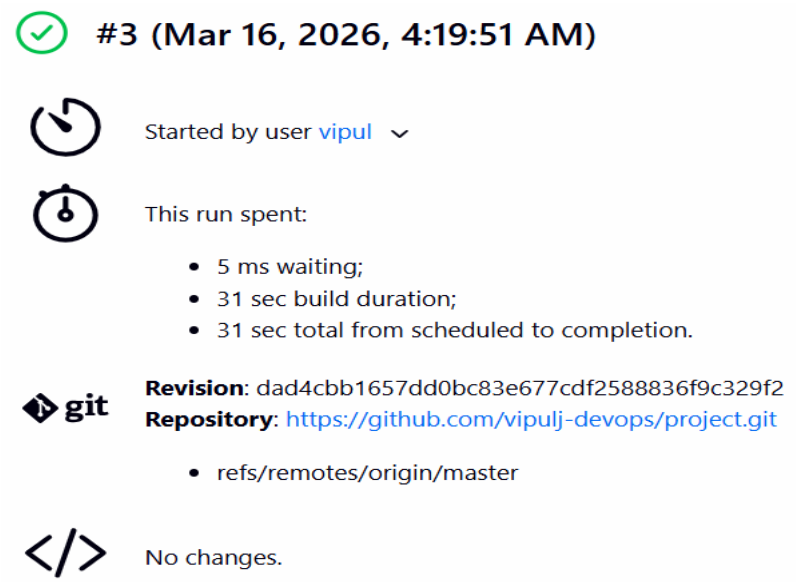
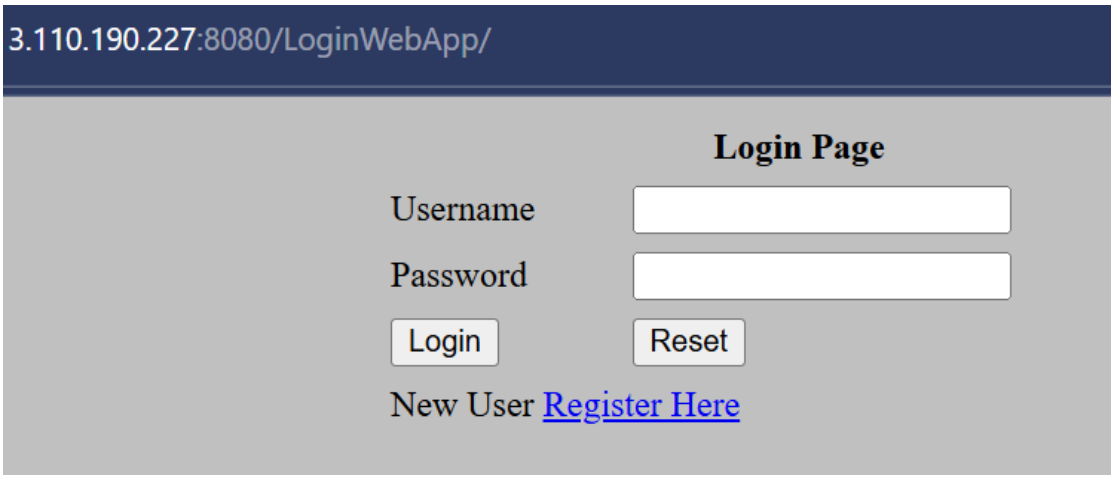
```
#!/bin/bash
SERVER_IP="3.110.190.227"
KEY="/mnt/other/mykey.pem"
WAR_FILE="/mnt/projects/target/LoginWebApp.war"
TOMCAT_LINK="https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.52/bin/apache-tomcat-10.1.52.zip"

echo "Copying WAR file to remote server..."
scp -i $KEY -o StrictHostKeyChecking=no $WAR_FILE ec2-user@$SERVER_IP:/home/ec2-user/

echo "Installing Tomcat and Deploying WAR..."
ssh -i $KEY -o StrictHostKeyChecking=no ec2-user@$SERVER_IP << EOF

echo "Installing Java..."
sudo yum install java-17-amazon-corretto -y

echo "Creating directory..."
sudo mkdir -p /mnt/servers
cd /mnt/servers

echo "Downloading Tomcat..."
sudo wget $TOMCAT_LINK

echo "Extracting Tomcat..."
sudo unzip apache-tomcat-10.1.52.zip

echo "Starting Tomcat..."
sudo /mnt/servers/apache-tomcat-10.1.52/bin/startup.sh

echo "Deploying WAR..."
sudo rm -rf /mnt/servers/apache-tomcat-10.1.52/webapps/LoginWebApp*
sudo cp /home/ec2-user/LoginWebApp.war /mnt/servers/apache-tomcat-10.1.52/webapps/
EOF

echo "Deployment finished"
```

Basically, the shell script automates the process of installing Apache Tomcat on a remote EC2 instance and deploying a WAR file from the Jenkins Master server. First, the script defines variables such as the remote server IP address, the PEM key used for SSH authentication, the location of the WAR file on the Jenkins server, and the download link

for Apache Tomcat. The script then uses the scp command to securely copy the WAR file from the Jenkins Master to the remote EC2 instance.

After transferring the WAR file, the script connects to the remote server using SSH and executes multiple commands remotely. These commands install Java (required for Tomcat), create the /mnt/servers directory, download the Tomcat package from the official Apache repository, and extract it. Once Tomcat is installed, the script starts the Tomcat server using the startup.sh script. Finally, the WAR file is copied into the Tomcat webapps directory, allowing Tomcat to automatically deploy the application. This script demonstrates automated remote server setup and application deployment using Jenkins

16. Job ran successfully



17. War file got added to webapp directory of tomcat in remote server and war file got deployed in tomcat and website got hosted